

Project: SecondSelf — Your Personal AI Second Brain

ProblemStatement.md

Every notes app fails the same way: you capture hundreds of notes, bookmarks, PDFs, and ideas — and then you never find them again. Information goes in, but nothing comes back out. Notes sit in folders nobody re-reads. Bookmarks pile up unread. Knowledge doesn't compound.

Goal: Build an end-to-end system where you can capture *anything* (a note, a link, a file), have AI automatically classify and file it, auto-link it to related knowledge, render it as a live interactive graph you can explore, and — most importantly — *ask it any question in plain English and get an answer synthesized from your own accumulated knowledge*. Then deploy it to a public URL anyone can open.

Not a notes app. Not a chatbot. A brain that organizes itself and answers for you.

Final System (what you're building over 4 weeks)

Capture any note/link/file



AI classifies & files it (PARA method)



AI auto-links it to related notes (embeddings)



Everything renders as a live, interactive, hoverable graph



Ask it anything in plain English → answer pulled from YOUR notes



Deployed on a public URL anyone can open

Week-by-Week Problem Statements

Each week is a self-contained problem. Build it, test it on **real** data (your own notes — not test data), and each week's output becomes the next week's input.

Week 1 — The Archivist: "Capture Everything, Lose Nothing"

Problem

You have no single place to put things. Ideas, links, and notes scatter across apps, browser tabs, and your memory. Build the foundation: **one command that captures anything into one place.**

Build

1. Set up the project structure from scratch:
 - raw/ — where every raw capture lands
 - wiki/ — (used later) organized, linked notes
2. Write a Python capture script that takes any note, link, or file and saves it into raw/ with:
 - a **timestamp**
 - a **unique ID**
 - the raw content
3. Test it on **10+ real** pieces of your own scattered information.

Deliverable ("Ship the Capture Pipeline")

- A working capture script — one command saves anything to raw/ with timestamp + unique ID.
- Your raw/ folder populated with **10+ real captured items** (not test data).
- 🏆 **Badge: The Archivist**

Acceptance Criteria

- [] raw/ and wiki/ folder structure exists
- [] One command captures a note, a link, AND a file
- [] Every capture has a timestamp + unique ID
- [] 10+ real items captured

Week 2 — The Librarian: "Teach AI to Organize For You"

Problem

A pile of raw captures is still a mess. Manual tagging never happens. Make the AI do the filing — and make it notice when two notes are about the same thing and link them automatically.

Build

2.1 — Auto-Classify (The Sorting Hat)

- Write a function that sends any raw capture to a free LLM (Groq / Llama 3) and gets back:
 - a **category** (using the **PARA** framework: Projects, Areas, Resources, Archives)
 - **tags**
 - a **one-line summary**
- Run it across last week's real captures and watch them organize themselves.

2.2 — Auto-Link Related Notes (Connect the Dots)

- Compute **embeddings** for each note (sentence-transformers, local + free).
- Compare each new capture against existing notes in wiki/.
- When content is related (similarity above a threshold), **auto-insert a link** between them.
- No manual tagging — the system notices relationships on its own.

Deliverable ("Ship the Self-Organizing Wiki")

- A pipeline that auto-classifies raw captures with PARA and auto-links related notes.
- Run on **15+ real items** → an organized wiki/ folder with linked notes.
- 🏆 **Badge: The Librarian**

Acceptance Criteria

- [] Any raw capture → category + tags + summary automatically
 - [] PARA categorization working
 - [] Embeddings computed per note
 - [] Related notes auto-linked (no manual tagging)
 - [] Runs on 15+ real items → organized wiki/
-

Week 3 — The Cartographer: "Visualize the Brain"

Problem

Your knowledge is now organized and linked — but you can't see it. Turn the wiki into something you can actually look at, explore, and watch think.

Build

3.1 — Graph Data Model (Give It a Shape)

- Write a script that reads every note and its links.
- Build a **nodes-and-edges** representation in memory:
 - every note → a **node**
 - every relationship/link → an **edge**
- Export it as clean **JSON**.

3.2 — Interactive Graph (The Brain Comes Alive)

- Use a JS graph library (**vis-network** or **Cytoscape.js**) to render:
 - notes as nodes (that pulse / are visually alive)
 - links as edges
 - **hover popups** that reveal each note's content
 - **drag-to-explore** and **zoom**
- A **force-directed** graph of your own knowledge.

Deliverable ("Ship the Living Brain")

- Your wiki converted to a graph and rendered as an interactive visual brain (hover, drag, zoom), built from your real notes.
- 🏆 **Badge: The Cartographer**

Acceptance Criteria

- [] Script builds nodes + edges from notes and exports clean JSON
- [] Interactive force-directed graph renders from that JSON
- [] Hover reveals note content
- [] Drag + zoom work
- [] Built from your real notes, not dummy data

Week 4 — The Oracle: "Ask It Anything, Ship It Public"

Problem

A visual brain is beautiful, but the real payoff is **answers**. Wire up natural-language search over everything you know — then package the whole thing into one deployable product.

Build

4.1 — Ask Your Brain (Natural Language Search)

- Build a single ask() function that combines:
 - the **embeddings** (find relevant notes to a question)
 - the **wiki** (the source content)
 - an **LLM** (synthesize an answer from retrieved notes)
- This is retrieval-augmented Q&A over your own knowledge.
- Test against real questions about your own captured notes.

4.2 — UI, Deployment, Public URL (Give It a Face)

- Assemble everything into **one Streamlit app**:
 - the interactive brain graph
 - the ask-anything search bar
- Deploy to a free platform (**Streamlit Cloud / HF Spaces**).
- Get a **public URL** anyone can open.

Deliverable ("Ship SecondSelf" — the final product)

Deploy the complete system — capture → auto-classify → auto-link → live interactive graph → ask-anything search — all wired into one Streamlit app with a public URL.

- 🏆 **Badge: The Oracle**

Acceptance Criteria

- [] ask() returns answers synthesized from your own notes (retrieval + LLM)
- [] One Streamlit app contains both the graph and the search bar
- [] Deployed live with a public URL
- [] Full pipeline works end to end in the deployed app

Final Deliverables (whole project)

- [] Public **GitHub repo** with a clean README + setup instructions
- [] **Live deployed URL** — interactive graph + ask-your-brain search, both working
- [] End-to-end flow verified: capture → classify → link → graph → ask
- [] All 4 weekly milestones complete (Capture Pipeline, Self-Organizing Wiki, Living Brain, SecondSelf deployment)

Suggested Repo Structure

```
secondself/
├── raw/           # Week 1: raw captures (timestamp + unique ID)
├── wiki/          # Week 2: classified + auto-linked notes
├── capture.py     # Week 1: one-command capture
├── classify.py     # Week 2: PARA classification via LLM
├── link.py        # Week 2: embeddings + auto-linking
├── build_graph.py # Week 3: nodes/edges → graph.json
├── graph.json     # Week 3: exported graph data
├── ask.py         # Week 4: retrieval + LLM answer
├── app.py        # Week 4: Streamlit UI (graph + search)
├── requirements.txt
└── README.md
```

Suggested Build Order in Cursor

1. Scaffold repo structure + requirements.txt
2. capture.py → test on real items (Week 1)
3. classify.py → PARA categories/tags/summary (Week 2.1)
4. link.py → embeddings + similarity auto-linking (Week 2.2)
5. build_graph.py → JSON nodes/edges (Week 3.1)
6. Graph render with vis-network/Cytoscape (Week 3.2)
7. ask.py → retrieval-augmented Q&A (Week 4.1)
8. app.py → Streamlit app combining graph + search (Week 4.2)
9. Deploy to Streamlit Cloud / HF Spaces → public URL
10. Write README, push to GitHub

AI Tools

- <https://cursor.com/download>
- <https://antigravity.google/product/antigravity-ide>
- <https://qoder.com/>
- <https://devin.ai/download/>
- VS code + Claude

Obsidian link - <https://obsidian.md/download>

Steps

Context

- problemStatement.txt → store the problem that you are solving
- architecture.md → HOW we will build the project
- implementation-plan.md → phase-wise implementation plan
 - Phase 0 → setup
 - Phase 1-5 → Implement code of the project
 - Phase 6-7 → locally test
 - Phase 8-9 → deploy the project + final round of testing
- edge-case.md → corner scenarios and edge-case

Prompts

1. Generate a detailed architecture for making this project using @PROBLEM_STATEMENT.md
2. Save the architecture into architecture.md file
3. Generate a phase-wise Implementation-plan.md using @architecture.md and @PROBLEM_STATEMENT.md
4. Generate an edge-case.md for storing all corner scenario and edge-cases for this project using @docs/architecture.md and @docs/Implementation-plan.md
5. Implement phase0 as per the @docs/Implementation-plan.md
6. Implement phase1 as per the @docs/Implementation-plan.md
7. URL
Push the project to github
make sure all unnecessary files are not pushed
8. Implement Sub-Phase 2.1 using the @docs/Implementation-plan.md
9. Implement Sub-Phase 2.2 using the @docs/Implementation-plan.md
10. I want to view embeddings that are stored in data folder
file name - embeddings.pkl
Write a script to show me embeddings in the file
11. Implement Sub-Phase 3.1 as per the @docs/Implementation-plan.md
12. Implement Sub-Phase 3.2 as per the @docs/Implementation-plan.md
13. stop the project
14. Implement Sub-Phase 4.1 as per the @docs/Implementation-plan.md
15. Implement Sub-Phase 4.2 as per the @docs/[Implementation-plan.md](#)
Deployment changes we can do in next week currently all local changes we can do
16. Stop the project
17. Generate a deployment-plan.md for deploying this project on streamlit

Implementation plan

SecondSelf — Phase-Wise Implementation Plan

A step-by-step build guide for SecondSelf, derived from PROBLEM_STATEMENT.md and architecture.md. Each phase is self-contained, testable on **real data**, and its output feeds the next phase.

How to Use This Document

1. Complete phases **in order** — do not skip ahead.
2. Test every phase on **your own notes, links, and files** (not dummy data).
3. Check off acceptance criteria before moving to the next phase.
4. Each phase ends with a **ship checkpoint** — a working artifact you can demo.

Total timeline: 4 weeks (one phase per week).

Phase Overview

Phase	Name	Badge	Primary Output
0	Foundation	—	Repo scaffold, deps, shared libs
1	The Archivist	 The Archivist	capture.py + 10+ items in raw/
2	The Librarian	 The Librarian	Classified + linked wiki/ (15+ items)
3	The Cartographer	 The Cartographer	graph.json + interactive graph
4	The Oracle	 The Oracle	ask() + Streamlit app on public URL

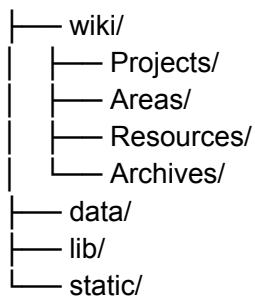
Phase 0 → Phase 1 → Phase 2 → Phase 3 → Phase 4
scaffold capture classify graph RAG +
raw/ link wiki visualize deploy

Phase 0 — Foundation (Day 0)

Goal: Scaffold the repo so every later phase has a consistent home for data and shared code.

Tasks

[] **0.1** Initialize git repo and create folder structure:
secondself/
├── raw/



[] 0.2 Create requirements.txt:

```
streamlit>=1.32
groq>=0.4
sentence-transformers>=2.3
numpy>=1.24
pyyaml>=6.0
pypdf>=4.0
requests>=2.31
beautifulsoup4>=4.12
python-dotenv>=1.0
```

[] 0.3 Create virtual environment and install dependencies:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

[] 0.4 Create .env.example and .gitignore:

```
# .env.example
GROQ_API_KEY=your_key_here
# .gitignore (minimum)
.venv/
.env
data/embeddings.pkl
__pycache__/
*.pyc
.DS_Store
```

[] 0.5 Implement lib/models.py — shared dataclasses:

Model	Fields
CaptureMeta	id, timestamp, type, source, original_filename, content_hash
CaptureResult	id, path, type

WikiNote	id, raw_id, para, tags, summary, created, links, body
GraphNode	id, label, para, tags, summary, content_preview, group
GraphEdge	source, target, weight, type
AskResult	answer, sources

•

[] **0.6** Implement lib/storage.py — filesystem helpers:

Function	Purpose
generate_capture_id()	{YYYY-MM-DD}_{uuid8}
write_raw_capture(meta, content)	Create raw/{id}/ folder
read_raw_captures()	List all unprocessed raw items
write_wiki_note(note)	Write wiki/{para}/{id}.md with YAML frontmatter
read_wiki_notes()	Parse all wiki markdown files
load_index() / save_index()	Read/write data/index.json
content_hash(data)	SHA-256 for dedup / change detection

[] **0.7** Initialize data/index.json:

```
{
  "raw_processed": {},
  "embeddings_version": "all-MiniLM-L6-v2",
  "last_graph_build": null
}
```

•

Verification

- [] python -c "from lib import models, storage" runs without error
- [] All directories exist; wiki/ has four PARA subfolders

Deliverable

Repo scaffold with shared models and storage layer — no user-facing features yet.

Phase 1 — The Archivist (Week 1)

Goal: One command captures any note, link, or file into raw/ with timestamp + unique ID.

Badge: 🏆 The Archivist

Tasks

- [] 1.1 Implement capture.py core functions:

Function	Input	Output
capture_note(text)	Plain text / markdown string	CaptureResult
capture_link(url, notes="")	URL + optional notes	CaptureResult
capture_file(path)	Path to local file	CaptureResult

- Each function must:
 - Generate ID via generate_capture_id()
 - Record ISO timestamp
 - Write raw/{id}/meta.json
 - Write content file (content.md, content.txt, or content.{ext})
 - Print confirmation: Captured → raw/2026-07-06_a1b2c3d4

[] 1.2 Implement CLI with argparse:
python capture.py note "Remember to review embeddings paper"
python capture.py link "https://arxiv.org/abs/..."
python capture.py file ./documents/resume.pdf
python capture.py # interactive stdin mode

- [] 1.3 Handle edge cases:

Case	Behavior
File does not exist	Print error, exit code 1

Empty note text	Reject with message
Binary file	Copy as-is; record original_filename in meta
Duplicate content	Warn (hash check); still allow capture

-

[] **1.4 Capture 10+ real items** from your own scattered information:

Suggested mix:

- 4–5 text notes (ideas, todos, journal snippets)
- 3–4 bookmarks (articles, docs, repos)
- 2–3 files (PDF, markdown doc, image with notes)

File Deliverables

File	Description
capture.py	CLI + capture functions
raw/{id}/meta.json	Metadata per capture
raw/{id}/content.*	Raw content per capture

Acceptance Criteria

- [] raw/ and wiki/ folder structure exists
- [] One command captures a note, a link, AND a file
- [] Every capture has a timestamp + unique ID
- [] 10+ real items captured (not test data)

Ship Checkpoint

Demo: capture three types in one session

python capture.py note "Career goal: transition to ML engineering by Q4"

python capture.py link "https://huggingface.co/sentence-transformers"

python capture.py file ~/Downloads/some-paper.pdf

ls raw/ # should show 10+ folders

Phase 2 — The Librarian (Week 2)

Goal: Auto-classify raw captures with PARA + tags + summary, then auto-link related notes via embeddings.

Badge: 📖 The Librarian

Sub-Phase 2.1 — Auto-Classify (Days 1–3)

- [] 2.1.1 Sign up for Groq and add GROQ_API_KEY to .env
- [] 2.1.2 Implement lib/llm.py:

Function	Purpose
call_llm(prompt, system="")	Groq API wrapper with retry
classify_content(text)	Returns {para, tags, summary} JSON
synthesize_answer(context, question)	RAG answer (used in Phase 4)

- Model: llama-3.1-8b-instant
- [] 2.1.3 Implement text extraction helpers in lib/storage.py or lib/extract.py:

Source type	Extraction method
note	Read content.md directly
link	requests + beautifulsoup4 strip HTML; fallback to URL string
file (PDF)	pypdf text extraction; fallback to filename

[] 2.1.4 Implement classify.py:
For each raw/ item not in index.json["raw_processed"]:
extract text → classify_content() → write wiki/{para}/{id}.md → update index.json

Wiki note format:

```
---
id: a1b2c3d4
raw_id: 2026-07-06_a1b2c3d4
para: Projects
tags: [ml, career]
summary: "One-line summary"
created: 2026-07-06T22:30:00Z
links: []
---
```

{cleaned body content}

-

[] **2.1.5** Run classifier on all Week 1 captures:

python classify.py

- [] **2.1.6** Manually spot-check 5 notes — verify PARA categories make sense

Sub-Phase 2.2 — Auto-Link (Days 4–7)

- [] **2.2.1** Implement lib/embeddings.py:

Function	Purpose
load_model()	Load all-MiniLM-L6-v2 (cached)
embed_text(text)	Return 384-dim vector
cosine_similarity(a, b)	Similarity score
load_embeddings() / save_embeddings()	data/embeddings.pkl

[] **2.2.2** Implement link.py:

For each wiki note (new or changed):

embed(title + summary + body)

compare vs all existing embeddings

if similarity $\geq$ 0.75:

add to frontmatter links[]

append [[other-id]] in body (deduplicated)

save embedding to embeddings.pkl

- [] **2.2.3** Tune similarity threshold:

Thresh old	Effect
0.65	More links, some noise
0.75	Start here — balanced
0.80	Fewer, higher-confidence links

[] **2.2.4** Implement pipeline.py orchestrator:

python pipeline.py classify # *classify only*

python pipeline.py link # *link only*

python pipeline.py process # *classify + link*

-
- [] **2.2.5** Capture 5+ additional real items, run full pipeline → 15+ total in wiki/

File Deliverables

File	Description
lib/llm.py	Groq client + classify/synthesize
lib/embeddings.py	sentence-transformers wrapper
classify.py	Raw → wiki classifier
link.py	Embedding similarity linker
pipeline.py	Orchestration script
wiki/{para}/*.md	15+ classified notes
data/embeddings.pkl	Embedding index
data/index.json	Processing state

Acceptance Criteria

- [] Any raw capture → category + tags + summary automatically
- [] PARA categorization working (all four categories used appropriately)
- [] Embeddings computed per note
- [] Related notes auto-linked (no manual tagging)
- [] Runs on 15+ real items → organized wiki/

Ship Checkpoint

```
python pipeline.py process
find wiki/ -name "*.md" | wc -l    # ≥ 15
grep -r "\[\" wiki/ | head        # wikilinks present
```

Open 2–3 linked notes side by side — confirm the connections are meaningful.

Phase 3 — The Cartographer (Week 3)

Goal: Convert the linked wiki into a force-directed interactive graph you can explore.

Badge: 🏆 The Cartographer

Sub-Phase 3.1 — Graph Data Model (Days 1–3)

- [] **3.1.1** Implement `build_graph.py`:

Step	Logic
Parse nodes	One node per <code>wiki/**/*.md</code>
Parse edges	From <code>links[]</code> <code>frontmatter + [[id]]</code> in body
Deduplicate	Edge key = <code>(min(source,target), max(source,target))</code>
Enrich nodes	<code>label = summary</code> , <code>group = para</code> , <code>content_preview = first 200 chars</code>
Export	Write <code>data/graph.json</code>

- [] **3.1.2** Validate `graph.json` schema:

```
{
  "nodes": [{ "id", "label", "para", "tags", "summary", "content_preview", "group" }],
  "edges": [{ "source", "target", "weight", "type" }],
  "metadata": { "generated_at", "node_count", "edge_count" }
}
```

-

- [] **3.1.3** Run builder and inspect output:

```
python build_graph.py
```

```
python -c "import json; g=json.load(open('data/graph.json')); print(g['metadata'])"
```

-

Sub-Phase 3.2 — Interactive Graph (Days 4–7)

- [] **3.2.1** Create `static/graph.html` with **vis-network**:
 - Load `graph.json` (inline or fetch)
 - Force-directed layout (Barnes-Hut physics)
 - Node colors by PARA group
 - Hover tooltip: `summary + content_preview`
 - Drag + zoom enabled
 - Optional pulse animation on nodes

- [] **3.2.2** Physics config starting point:

```
physics: {
  barnesHut: { gravitationalConstant: -8000, springLength: 150 },
  stabilization: { iterations: 200 }
```

}

-

[] **3.2.3** Test graph standalone in browser:

python -m http.server 8000

Open *http://localhost:8000/static/graph.html*

-

- [] **3.2.4** Verify interactions:

Interacti on	Expected
Hover node	Tooltip with note summary
Drag node	Node moves, graph re-settles
Scroll	Zoom in/out
Click node	(Optional) highlight connected edges

-

- [] **3.2.5** Wire build_graph.py into pipeline.py process so graph rebuilds after linking

File Deliverables

File	Description
build_graph.py	Wiki → graph.json builder
data/graph.json	Exported graph data
static/graph.html	vis-network interactive viewer

Acceptance Criteria

- [] Script builds nodes + edges from notes and exports clean JSON
- [] Interactive force-directed graph renders from that JSON
- [] Hover reveals note content
- [] Drag + zoom work
- [] Built from your real notes, not dummy data

Ship Checkpoint

```
python build_graph.py
python -m http.server 8000
# Open graph in browser — explore your knowledge brain
```

Phase 4 — The Oracle (Week 4)

Goal: Ask questions in plain English, get answers from your notes, and deploy everything as a public Streamlit app.

Badge: 🏆 The Oracle

Sub-Phase 4.1 — Ask Your Brain (Days 1–3)

[] **4.1.1** Implement ask.py:

def ask(question: str, top_k: int = 5) -> AskResult:

- Pipeline:
 1. Embed question (lib/embeddings.py)
 2. Retrieve top-K notes by cosine similarity from embeddings.pkl
 3. Load full wiki bodies for retrieved IDs
 4. Build RAG prompt with note context
 5. Call synthesize_answer() via lib/llm.py
 6. Return { answer, sources: [{id, summary, relevance_score, para}] }

[] **4.1.2** RAG prompt template:

You are SecondSelf, answering from the user's personal knowledge base.

Use ONLY the provided notes. If the answer isn't in the notes, say so.

Cite sources as [note-id].

Notes:

{retrieved_notes}

Question: {question}

- [] **4.1.3** Guardrails:

Setting	Value
top_k	5 (default)
Temperature	0.3
Max context	~6000 tokens (truncate long notes)
No relevant notes	Return "I don't have notes about that"

[] 4.1.4 Test with 5+ real questions about your captured notes:
python ask.py "What are my career goals?"
python ask.py "What ML resources have I saved?"
python ask.py "Summarize my active projects"

Sub-Phase 4.2 — Streamlit App + Deployment (Days 4–7)

[] 4.2.1 Implement app.py layout:

 SecondSelf	[Refresh Graph]	
Ask your brain: [_____]	[Ask]	
Answer panel + source citations		
Interactive Knowledge Graph (vis-network)		
Sidebar: Capture Process Stats		

[] 4.2.2 Wire components:

UI Element	Backend
Ask bar	ask.ask(question)
Graph	st.components.v1.html() embedding vis-network
Capture form	capture.capture_note(text)
Process button	pipeline.process() → rebuild graph
Stats sidebar	Count nodes/edges from graph.json
Refresh button	Re-run build_graph.py, reload component

[] 4.2.3 Add caching:
@st.cache_resource
def load_embeddings(): ...

@st.cache_data
def load_graph(): ...

-
- [] **4.2.4** Test locally:
streamlit run app.py
 - Verify full flow: capture → process → graph updates → ask returns answer.
 - [] **4.2.5** Write README.md:
 1. Project description + screenshot
 2. Setup instructions (venv, deps, .env)
 3. Usage (capture, process, ask)
 4. Architecture overview (link to architecture.md)
 5. Live demo URL
 - [] **4.2.6** Push to GitHub (public repo)
 - [] **4.2.7** Deploy to **Streamlit Community Cloud**:
 1. Connect GitHub repo at share.streamlit.io
 2. Set main file: app.py
 3. Add secret: GROQ_API_KEY
 4. Pre-commit wiki/, data/graph.json, data/embeddings.pkl for demo
 5. Verify public URL loads
 - [] **4.2.8** End-to-end test on deployed app:

S	Action
1	Open public URL
2	Graph renders with real nodes
3	Ask a question → get synthesized answer
4	Capture a new note via sidebar
5	Process → graph updates

File Deliverables

File	Description
ask.py	RAG Q&A engine
app.py	Streamlit UI
README.md	Setup + usage docs

Live URL <https://secondself-{user}.streamlit.app>

Acceptance Criteria

- [] ask() returns answers synthesized from your own notes (retrieval + LLM)
- [] One Streamlit app contains both the graph and the search bar
- [] Deployed live with a public URL
- [] Full pipeline works end to end in the deployed app

Ship Checkpoint

Share your public URL. Demo: ask a question → see answer with sources → explore the graph.

Final Integration Checklist

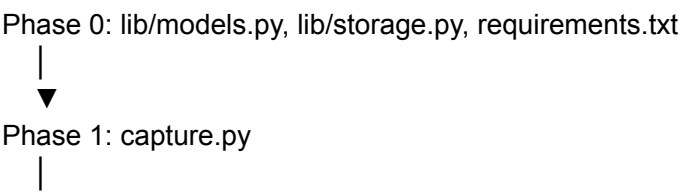
Before calling the project complete, verify the full pipeline:

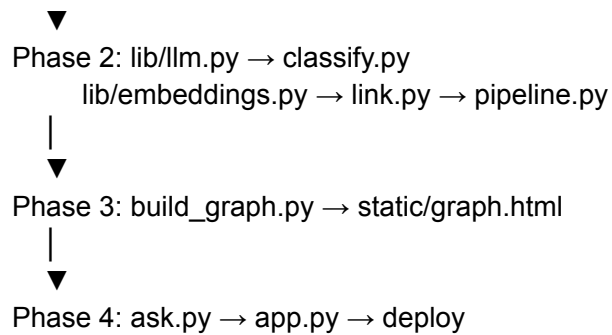
Capture → Classify → Link → Graph → Ask → Deploy

- [] Public **GitHub repo** with clean README + setup instructions
- [] **Live deployed URL** — interactive graph + ask-your-brain search, both working
- [] End-to-end flow verified in production
- [] All 4 weekly milestones complete:

Milestone	Status
Capture Pipeline (Phase 1)	☐
Self-Organizing Wiki (Phase 2)	☐
Living Brain (Phase 3)	☐
SecondSelf Deployment (Phase 4)	☐

Dependency Map (Build Order)





Risk Register

Risk	Phase	Mitigation
Groq API rate limits	2, 4	Batch classify; add retry with backoff
Embedding model slow on first load	2, 4	@st.cache_resource in Streamlit
Too many / too few auto-links	2	Tune threshold (0.65–0.80)
PDF text extraction fails	2	Fallback to filename; store raw file
Graph too cluttered	3	Filter by PARA category; limit edge weight display
Private notes on public URL	4	Use demo-safe data or document the tradeoff
Streamlit iframe sizing	4	Set explicit height on st.components.v1.html()

Quick Reference — Commands

Phase 1

```
python capture.py note "..."  
python capture.py link "https://..."  
python capture.py file ./doc.pdf
```

Phase 2

```
python classify.py  
python link.py  
python pipeline.py process
```

Phase 3

```
python build_graph.py
```

```
python -m http.server 8000 # preview graph
```

Phase 4

```
python ask.py "What are my career goals?"
```

```
streamlit run app.py
```

References

- PROBLEM_STATEMENT.md — weekly goals and acceptance criteria
- architecture.md — data models, component design, deployment